

Oscilloszkóp, FFT vizualizációval STM32F429I-DISC1 fejlesztőkártyára

Áttekintés

Kedves Olvasó!

Ebben a dokumentumban olvashatja az *ARM Cortex magú mikrovezérlők (BMEVIMIAV07)* tárgy házi feladat megoldásomnak dokumentációját.

A házi feladat megoldása az *STM32F429I-DISC1* fejlesztőkártyán készült el, ami az „*STM32F429ZIT6*” típusnevű, ARM Cortex-M4 architektúrájú, lebegőpontos számítási egységgel (FPU) ellátott processzort tartalmazza.

Cél / Használati utasítás

A projekt célja egy grafikus felhasználói felülettel ellátott egysatornás oszcilloszkóp/*FFT* kijelző létrehozása, amivel audio jeleket lehet ábrázolni. A program különböző funkciói a fedélzeti gomb és az érintőképernyő használatával érhetőek el.

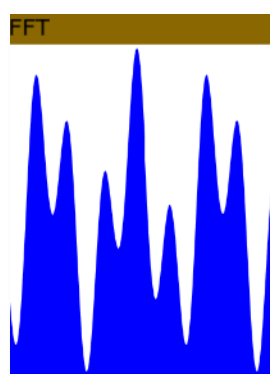
A **fedélzeti gomb (PA0)** megnyomásával a felhasználó az oszcilloszkóp (Scope) és *FFT* vizualizáció funkciók között válthat.

A „*Scope*” funkción belül, a fenti sávban található **kék** gombra kattintás esetén az oszcilloszkóp nagyítása beállítható *egyszeres* / *kétszeres* / *négyszeres* opciókra. Egyszeres nagyításnál a fehér terület alját érintő görbe a 0 Voltot, a fehér terület tetejét pedig a 3 Voltos feszültséget jelenti.¹

Előnézet



1. ábra – Oszcilloszkóp minta



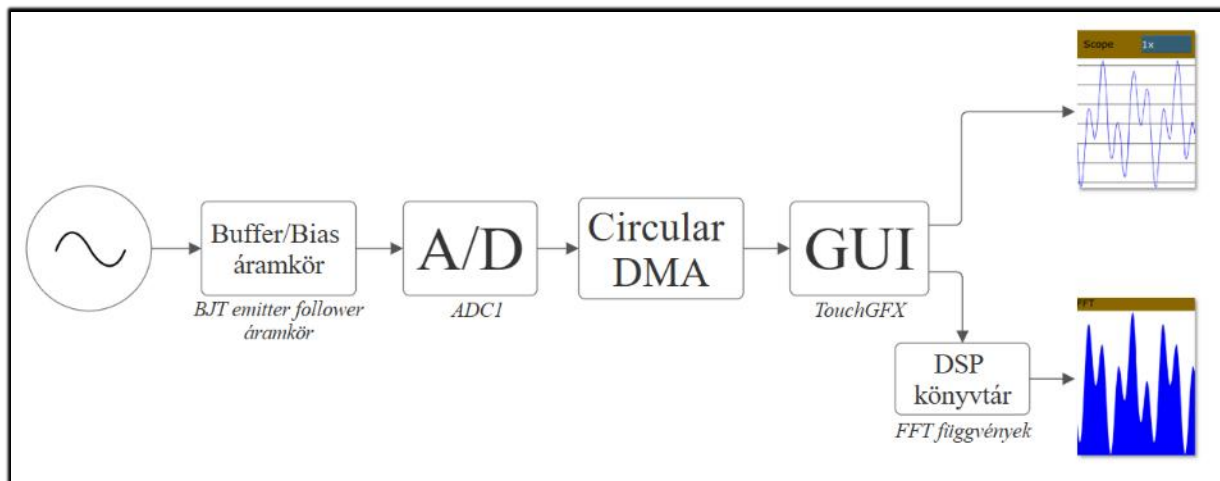
2. ábra - FFT vizualizáció minta

¹ Megjegyzés: A későbbiekben ezen értékeket szemantikai okokból érvénytelennek fogjuk tekinteni, hiszen a digitális átalakításnál az A/D konverter nem tud negatív feszültségekkel dolgozni, ezért a bemeneti jelünket elő kell feszíteni 1.5V-al, hogy az analóg bemenet -1.5V és +1.5V feszültségek között feldolgozható és kimutatható lehessen.

Tartalomjegyzék

Áttekintés	1
Cél / Használati utasítás	1
Előnézet	1
Tartalomjegyzék	2
Működési blokkdiagram	3
Hardveres és szoftveres konfiguráció	3
Pinek	3
Órajelek	4
Konstansok	4
Digitális mintavételezés	5
Elmélet	5
Implementáció	5
ADC	5
DMA	7
Analóg frontend	7
Fast Fourier Transform	8
Elmélet	8
Fourier-transzformáció	8
DFT	8
FFT	8
Implementáció	8
Felhasználói felület	9
Működési elv	9
Implementáció	10
Scope Screen	10
FFT Screen	11
Eredmények	12
Tesztelés különböző formájú jelekkel	12
Felhasznált szoftver és hardver	13
Szoftverek	13
Hardverek	13
Elektronikai alkatrészek	13

Működési blokkdiagram



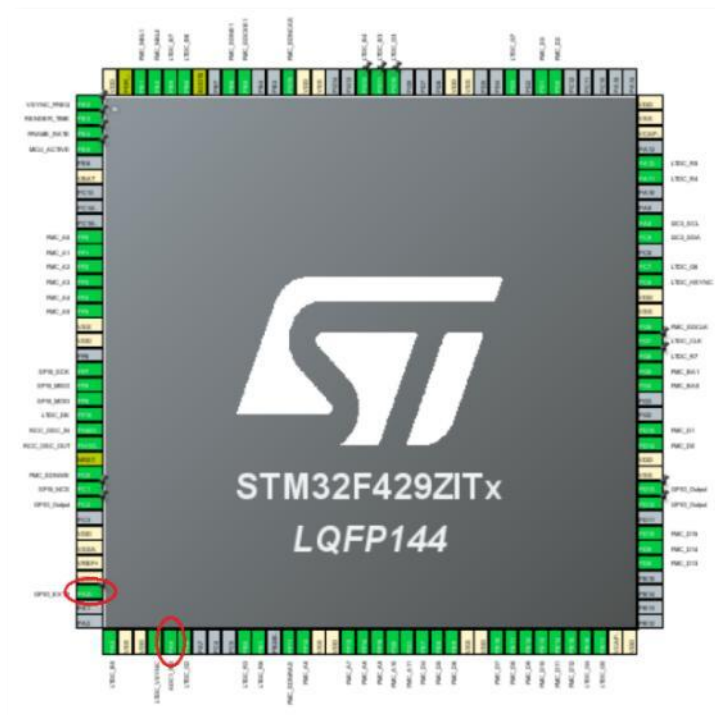
3. ábra - Program blokkdiagramja

Hardveres és szoftveres konfiguráció

A hardver alapkonfigurációja a TouchGFX „STM32F429 Discovery Kit” sablon projekt által generált .ioc fájl szerintiek. Bármilyen külső változtatás piros körrel van jelölve.

Pinek

A kézzel konfigurált pinek a PA0 és a PA5-ös pinek, amik rendre a nyomógomb „GPIO-EXTIO” interruptjára és az ADC1 „ADC1-IN5” bemenetére lettek konfigurálva.



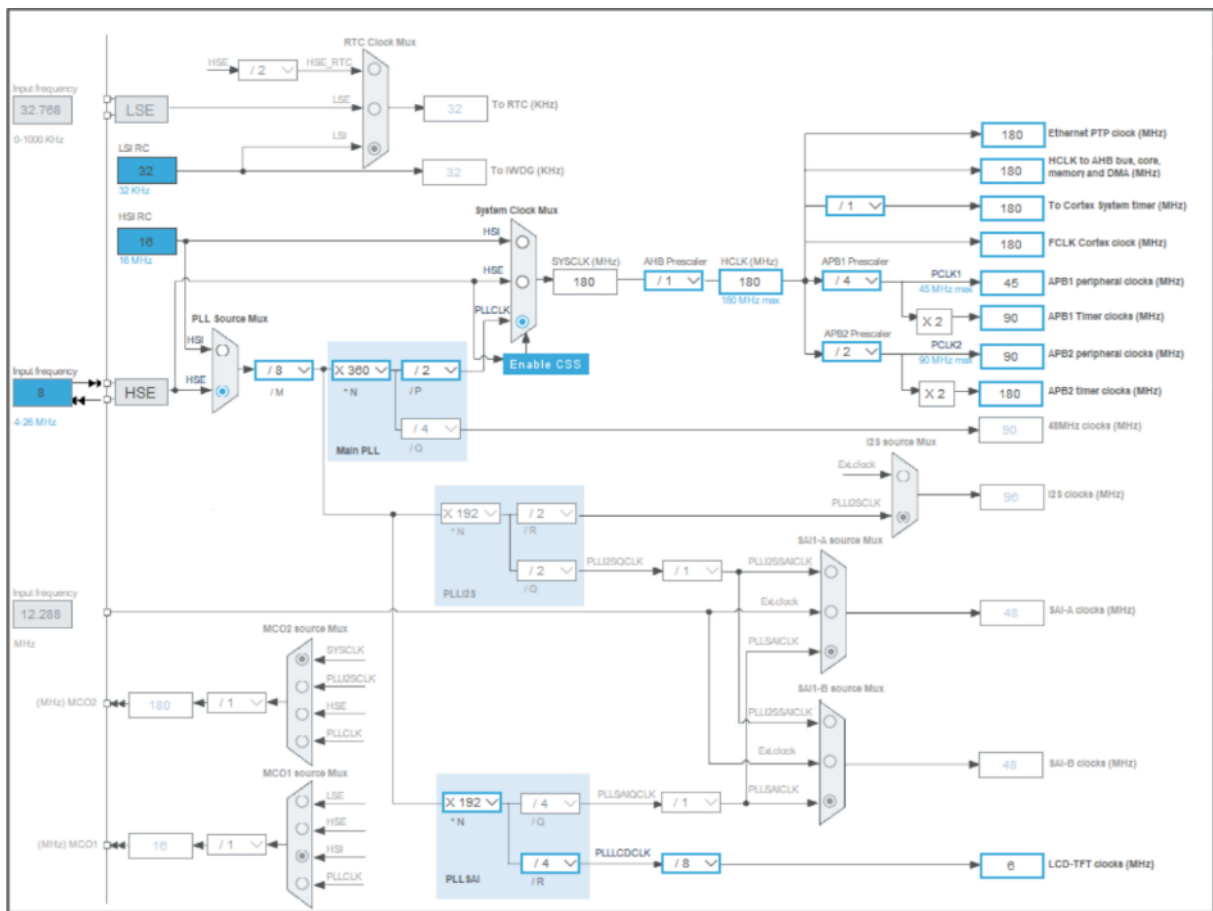
4. ábra - Pin konfiguráció

Órajelek

Az órajelek konfiguráció teljes mértékben a TouchGFX „STM32F429 Discovery Kit” sablonprojekt szerintiék, itt kézi változtatás nem történt.

Fontosabb számok:

- SYSCLK: 180 MHz
- PCLK2: 90 MHz



5. ábra - Órajelek konfiguráció

Konstansok

A forráskód *main.h* fájljában vannak definiálva a program optimális működéséhez beállított konstansok.

```
1. #define ADC_BUFFER_SIZE 256
2. #define FFT_BUFFER_SIZE 4096
3. #define FFT_SAMPLE_RATE 45500
4. #define FFT_BINS 240
```

- ADC_BUFFER_SIZE: A DMA által feltöltött A/D mintavételek bufferje.
- FFT_BUFFER_SIZE: Az FFT kiszámítására összegyűjtött minták ablaka.
- FFT_SAMPLE_RATE: A megfelelő kirajzoláshoz tartozó mintavételi ráta.
- FFT_BINS: A vizuálisan látható FFT binek száma. (Nem összekeverendő az FFT számára elérhető ablakmérettel, ami az FFT_BUFFER_SIZE)

Digitális mintavételezés

Elmélet

Ahhoz, hogy az analóg bemeneti jelünket digitálisan feldolgozni tudjuk, A/D átalakítást kell végeznünk.

A digitális mintavételezés alaptétele a „Nyquist-tétel”, amely szerint a mintavételezési ráta legyen nagyobb, mint a feldolgozni kívánt legmagasabb frekvencia kétszerese.

$$f_{\text{sampling}} > 2f_{\text{max}}$$

A használt kártyán max. 12-bites „szukcesszív approximációs” átalakítók vannak, azaz a bemeneti jeleket a bináris keresés algoritmus segítségével sorolja be diszkrét értékekre. Ez lesz a mintavételünk. A mellékelt ábrán (6. ábra) látható az A/D átalakító blokkdiagramja.

Azt szeretnénk, hogy az A/D átalakító folyamatosan mintavételezzon, illetve ebből kifolyólag azt is, hogy a processzornak ne kelljen állandóan a mintákat az egyik helyről a másikra másolni. Ehhez használjunk DMA-t.

Implementáció

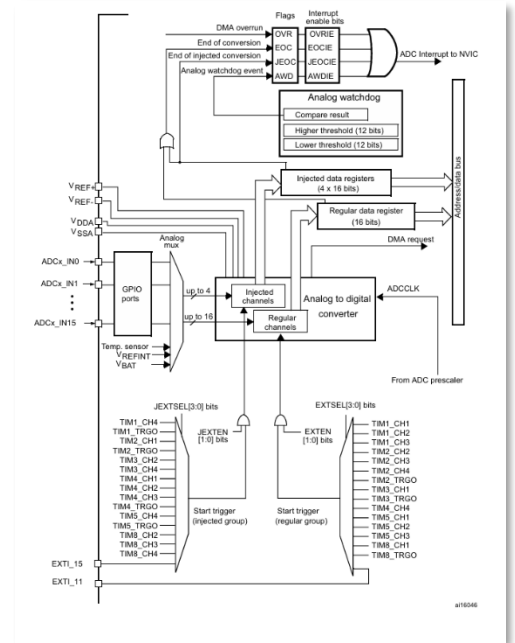
ADC

- A megvalósításhoz az ADC1 modult engedélyeztem 12-bites módban, folytonos mintavételezéssel, és a folytonos DMA-kérések engedélyezésével. Az ADC órajele a PCLK2 jelforrás leosztva 4-gyel, ami így tehát: $\frac{90 \text{ MHz}}{4} = 22.5 \text{ MHz}$.

```
hadc1.Instance = ADC1;
hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV4;
hadc1.Init.Resolution = ADC_RESOLUTION_12B;
hadc1.Init.ScanConvMode = DISABLE;
hadc1.Init.ContinuousConvMode = ENABLE;
hadc1.Init.DiscontinuousConvMode = DISABLE;
hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
hadc1.Init.NbrOfConversion = 1;
hadc1.Init.DMAContinuousRequests = ENABLE;
hadc1.Init.EOCSelection = ADC_EOC_SINGLE_CONV;
if (HAL_ADC_Init(&hadc1) != HAL_OK)
{
    Error_Handler();
}
```

- Engedélyeztem a bemenetet az 5-ös csatornán, ami a PA5 pinnek felel meg:

```
sConfig.Channel = ADC_CHANNEL_5;
sConfig.Rank = 1;
sConfig.SamplingTime = ADC_SAMPLETIME_480CYCLES;
if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
{
    Error_Handler();
}
```



6. ábra - STM32F4xx család ADC blokkdiagramja

- Elindítottam a DMA átvitelt, ami a konfigurációnak megfelelően beteléskor újraindul:

```
if (HAL_ADC_Start_DMA(&hadc1, (uint32_t*)ADC_BUFFER, ADC_BUFFER_SIZE) != HAL_OK)
{
    Error_Handler();
}
```

A mintavételezési ráta kiszámolható úgy, hogy elosztjuk az ADC órajelét az ADC ciklusszám és a mintavételi ciklusszám összegével, azaz:

$$SR = \frac{CLK_{ADC}}{CYC_{ADC} + CYC_{SAMPLE}} = \frac{22.5 \text{ MHz}}{15 + 480} = \frac{22.5 \text{ MHz}}{495} \approx 45\,500 \text{ S/s}$$

Az alábbi Callback függvények lettek definiálva:

- *HAL_ADC_ConvHalfCpltCallback* / *HAL_ADC_ConvCpltCallback*: Flaget állít, majd az adatot továbbítja az FFT-buffernek, ha az FFT-mód aktív és a buffer készen áll.

```
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef* hadc)
{
    if (hadc->Instance == ADC1)
    {
        IsADCHalfReady = 0b1;

        if (IsFFTMMode && !IsFFTReady)
            PushToFFTBuf((uint16_t*) &ADC_BUFFER[0], ADC_BUFFER_SIZE / 2);
    }
}
```

```
void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef* hadc)
{
    if (hadc->Instance == ADC1)
    {
        IsADCFullReady = 0b1;

        if (IsFFTMMode && !IsFFTReady)
            PushToFFTBuf((uint16_t*) &ADC_BUFFER[ADC_BUFFER_SIZE / 2],
                        ADC_BUFFER_SIZE / 2);
    }
}
```

- Ha az FFT-bufferje készen áll a következő átvitelre, ez a függvény fut le:

```
int writeIndex = 0;
void PushToFFTBuf(uint16_t* src, int len)
{
    for(int i = 0; i < len; i++)
    {
        FFT_BUFFER[FFTwriteIndex++] = src[i];

        if(FFTwriteIndex >= FFT_BUFFER_SIZE)
        {
            FFTwriteIndex = 0;
            IsFFTReady = 0b1;
        }
    }
}
```

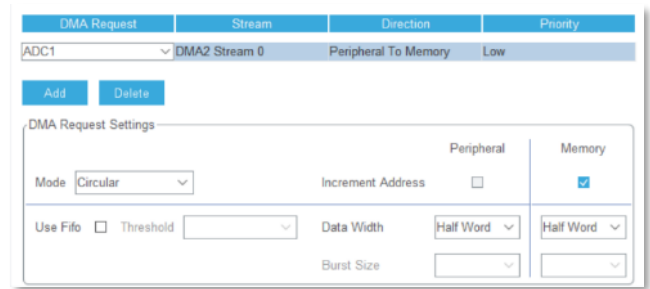
DMA

Az ábrán látható (7. ábra) DMA beállításnak köszönhetően elég a DMA-t egyszer elindítani, és a buffer megtelítése után a DMA automatikusan elkezd a következő átvitelt.

A DMA „half-word” módra van állítva, ami azt jelenti, hogy a DMA félhosszúságú kifejezésekkel működik, tehát 32-bites adatok helyett 16-biteseket vár.

➤ A DMA inicializáló kódja:

```
static void MX_DMA_Init(void)
{
    __HAL_RCC_DMA2_CLK_ENABLE();
    HAL_NVIC_SetPriority(DMA2_Stream0_IRQn, 5, 0);
    HAL_NVIC_EnableIRQ(DMA2_Stream0_IRQn);
}
```



7. ábra - ADC DMA átvitellel

Analóg frontend

A dokumentum első oldalán leírtak szerint az ADC nem képes negatív feszültségeket feldolgozni, tehát azokat nyersen nem lehet bemenetként adni az ADC-nek. Valahogyan el kell azt érni, hogy a maximálisan -1.5V és +1.5V között lévő analóg jel elő legyen feszítve 1.5 Volt-al, hogy az ADC bemenete 0 és 3 Volt között legyen.

Ehhez egy egyszerű bipoláris tranzistoros buffer áramkört használtam, amit az ábrán is láthatunk. (8. ábra) A BJT emitter follower áramkör kimeneti impedanciája „elegendően alacsony” ahhoz, hogy meg tudja hajtani a bemenetünket.

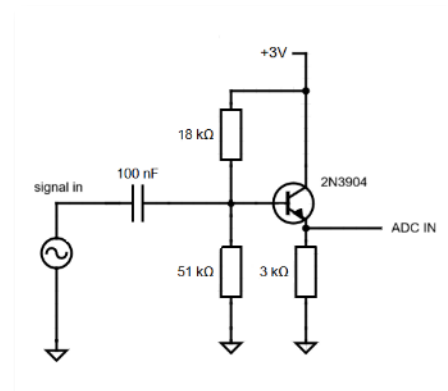
Méréseim alapján az általam használt 2N3904-es tranzisztor bázis-emitter átmenet nyitófeszültsége 0.68V.

Az emitter csomópontján 1.5 Volt-ot szeretnénk látni, tehát a bázison $1.5V + 0.68V = 2.18V$ DC feszültségnek kell lennie. A feszültségosztó formula szerint:

$$V_{ki} = V_{be} * \frac{R_2}{R_1 + R_2} = 3V * \frac{51k\Omega}{18k\Omega + 51k\Omega} = 2.217V.$$

A hiba mértéke ~1.7% felfelé. - „Jó lesz...”

A 37mV-os hibát szoftveresen fogjuk korrigálni egy korrekciós faktor használatával.



8. ábra - BJT emitter follower előfeszítő hálózat

Fast Fourier Transform

Elmélet

Fourier-transzformáció

A *Fourier-transzformáció* célja az, hogy valamilyen időtartománybeli jelet frekvenciatartományba alakítsunk, azaz lássuk, hogy az adott bemeneti jel milyen frekvenciákból tevődik össze.¹

Folytonos függvényekre a formula:

$$F(\omega) = \int_{-\infty}^{+\infty} f(t)e^{-j2\pi\omega t} dt$$

DFT

Diszkrét idejű és értékkészletű függvényekre a *Diszkrét Fourier-transzformáció*:

$$X(k) = \sum_{n=0}^{N-1} x(n)e^{-\frac{j2\pi nk}{N}}$$

Ez azonban lassú, ugyanis a kiszámítási bonyolultsága $O(n^2)$. Valami jobb kell...

FFT

A *Fast Fourier Transform*, vagy röviden *FFT*, a *Diszkrét Fourier-transzformáció* algoritmus gyorsabb és optimalizáltabb végrehajtására jött létre. Ennek az algoritmusnak többféle variációja is létezik, de talán a legnépszerűbb a *Cooley-Tukey-féle FFT*.

Az *FFT* algoritmus kiszámítási bonyolultsága $O(n \log n)$. Ennek az algoritmusnak a végrehajtása már realizisztikusabb, és valós időben is használható akár beágyazott hardveren is, minimális hibafaktoral.

Ez már 1024 pontnál is masszív különbségeket képez:

$$1024\text{-point DFT} \rightarrow O(1024^2) = 1\,048\,576$$

$$1024\text{-point FFT} \rightarrow O(1024 * \log_2 1024) = 10\,240$$

Az algoritmus működését itt nem részletezem, de a lábjegyzetben található linken utána tud nézni az olvasó.²

Implementáció

Az STM32CubeMX konfigurátor program tartalmaz opcionálisan telepíthető middleware-eket, ezek között szerepel a „X-CUBE-ALGOBUILD” könyvtár is, aminek a feltelepítését követően elérhetővé válik a DSP függvénykönyvtár.³ A könyvtár többek között tartalmazza az

¹ A Fourier-transzformáció a Laplace-transzformáció különleges esete, ahol a Laplace-transzformáció komplex paraméterének valós része nulla ($\sigma = 0$), tehát $s = \sigma + i\omega$ helyett $s = i\omega$. Ebből az következik, hogy a Fourier-transzformáció csak az oszcillációt vizsgálja, de a függvény exponenciális csillapodását, vagy növekedését nem.

² FFT-algoritmus működése: <https://pythonnumericalmethods.studentorg.berkeley.edu/notebooks/chapter24.03-Fast-Fourier-Transform.html>

³ FFT in real-time in STM32 MCUs?: <https://hackaday.io/page/38607-fft-in-real-time-in-stm32-mcus>

FFT algoritmus beágyazott hardverre optimalizált implementációját. Az *FFT*-vel kapcsolatos függvények a könyvtár „*arm_math.h*” header fájljában érhetőek el.

- Először példányosítani és inicializálni kell az *FFT*-számító egységet¹:

```
#include "arm_math.h"
arm_rfft_fast_instance_f32 engine;

.....
.....
.....

arm_rfft_fast_init_f32(&engine, FFT_BUFFER_SIZE);
for (int i = 0; i < FFT_BUFFER_SIZE; i++)
{
    window[i] = 0.5f * (1.0f - cosf(2.0f * PI * i / (FFT_BUFFER_SIZE - 1)));
}
```

- Majd, amikor összegyűlt a kellő mennyiségű sample az *FFT*-bufferbe:

```
for (int i = 0; i < len; i++)
{
    const int correction = 40; //Korrekciós faktor
    in[i] = (ptr[i] - 2048 - correction) / 2048.0f;
    in[i] *= window[i];
}

arm_rfft_fast_f32(&engine, in, out, 0);
arm_cmplx_mag_f32(out + 2, magnitudes + 1, (FFT_BUFFER_SIZE / 2) - 1);
magnitudes[0] = fabsf(out[0]); // DC tagot külön kezeljük le
```

Felhasználói felület

Működési elv

Beágyazott platformokon a grafikus felület fejlesztése nem könnyű feladat, hiszen rengeteg perifériát és órajelet kell megfelelően beállítani, például az *LTDC*-t, *DMA2D*-t, *FMC*-t. Sok hibalehetőség van, és bármilyen hiba után a fedélzeti LCD nem a kívánt módon működik.

A hibalehetőségek elkerülése érdekében a grafikus felhasználói felület implementálásához az STMicroelectronics által fejlesztett *TouchGFX* C++ keretrendszert használtam, aminek a segítségével könnyen és gyorsan lehet grafikus felületeket a *TouchGFXDesigner* WYSIWYG-editoron² keresztül létrehozni.

A *TouchGFX* keretrendszer a *Model, View, Presenter (MVP)* architektúrát használja, ami elválasztja az alkalmazás logikai kódját (Model) a megjelenítési rétegtől (View), és a kettő réteg között van egy köztes szereplő (Presenter), aki a két réteg közötti adatátvitelért felelős.

Az *TouchGFXDesigner*-ban a sablonprojekt létrehozása után lehetőségünk van rögtön kódot generáltatni, ami helyettünk elvégzi az adott fejlesztőkártyához szükséges perifériák konfigurálását, és a futtatás gombra kattintva képes a build-elt programot a flash-re írni.

¹ A for-ciklusban előre kiszámítunk egy „*Hann-ablakot*”, ami csökkenti a digitális jelek spektrális szivárgását az *FFT* futásakor.

² WYSIWYG: angol rövidítés, jelentése: **What You See Is What You Get**

Implementáció

A *TouchGFXDesigner*-ben létrehoztam egy „*STM32F429 Discovery Kit*” sablonprojektet, amin az első oldalon látható ábrák szerinti felhasználói felületeket alakítottam ki. (1. ábra) (2. ábra)

Scope Screen

A felső barna sávban helyet kapott a funkció felirata, és a nagyításra szolgáló gomb. A képernyő közepén látható a bemenet grafikonja.

A grafikon vízszintes tengelyén a bemeneti jelet ábrázolja az időtartományban.

A függőleges tengelyen ábrázolja az amplitúdót, amit -1.5 V és +1.5 V között jelez ki. A középső vastag vízszintes vonal jelenti a 0 V szintjét.

- A grafikon renderelése a következőképpen történik:

```
for (int i = 0; i < len; ++i)
{
    const int correction = 40;
    volatile int16_t val = ptr[i];
    float p = (val - 2048 - correction) / 2048.0f;
    scope.addDataPoint(p);
}

scope.invalidate();
```

- A **kék** gomb megnyomására a *TouchGFX* ezt a függvényt futtatja le:

```
unsigned scale = 0;
void ScopeScreenView::pointsButton_clicked()
{
    scope.clear();

    scale = (scale + 1) % 3;

    switch(scale)
    {
    case 0:
        scope.setGraphRangeY(-1.0f, 1.0f);
        pointsButton.setLabelText(touchgfx::TypedText(T_SCOPE_SCALE_UNITY));
        break;

    case 1:
        scope.setGraphRangeY(-0.5f, 0.5f);
        pointsButton.setLabelText(touchgfx::TypedText(T_SCOPE_SCALE_HALF));
        break;

    case 2:
        scope.setGraphRangeY(-0.25f, 0.25f);
        pointsButton.setLabelText(touchgfx::TypedText(T_SCOPE_SCALE_QUARTER));
        break;
    }

    pointsButton.invalidate();
}
```

FFT Screen

A felső barna sávban látható a funkció neve, a képernyő közepén pedig az *FFT*-grafikon.

A grafikon vízszintes tengelyén 50 és 20000 Hz között ábrázoljuk logaritmikus skálán a frekvenciákat.

A képernyő közepén megjelenő frekvenciaérték így tehát:

$$f_{\text{közép}} = \sqrt{50 \text{ Hz} * 20000 \text{ Hz}} = 1000 \text{ Hz}$$

A függőleges tengelyen pedig a jelablak amplitúdója látható dB FS-skálán. A függőleges tengely alján a -100 dB FS érték van megadva, a tetején pedig a 0 dB FS, tehát a maximálisan ábrázolható érték, vagyis a csúcstól-csúcsig 3 V-os jel.

➤ Az *FFT*-grafikon kirajzolása így történik:

```
void FFTScreenView::updateFFT(volatile uint16_t* ptr, int len)
{
    ProcessFFT(ptr, len);

    for (int x = 0; x < FFT_BINS; x++)
    {
        // Logarithmic bin selection
        float f0 = FFT_MIN_FREQ * powf(FFT_MAX_FREQ / FFT_MIN_FREQ, (float) x / (float) FFT_BINS);
        float f1 = FFT_MIN_FREQ * powf(FFT_MAX_FREQ / FFT_MIN_FREQ, (float)(x + 1) / (float) FFT_BINS);
        int bin0 = (int)(f0 * FFT_BUFFER_SIZE / FFT_SAMPLE_RATE);
        int bin1 = (int)(f1 * FFT_BUFFER_SIZE / FFT_SAMPLE_RATE);

        bin0 = fmax(0, bin0);
        bin1 = fmin(FFT_BUFFER_SIZE / 2 - 1, bin1);

        // RMS based weighted sum
        float sum = 0;
        for (int k = bin0; k <= bin1; k++)
        {
            sum += magnitudes[k] * magnitudes[k];
        }
        float val = sqrtf(sum / (bin1 - bin0 + 1));

        // Scaling according to the FFT buffer's size
        volatile float mag = (2.0f * val) / FFT_BUFFER_SIZE;

        // dBFS conversion
        volatile float dB = 20.0f * log10f(mag + 1e-6f);
        dB = fmaxf(dB, -100.0f);
        dB = fminf(dB, 0.0f);

        // Add +100 due to GUI limits (100 = full scale)
        fft.addDataPoint(dB + 100.0f);
    }

    fft.invalidate();
}
```

- Meghívja az *FFT* számítást végző függvényt
- logaritmikus frekvenciasávokra bontja a spektrumot
- RMS-alapú amplitúdóbecslést végez
- az amplitúdókat dBFS skálára konvertálja
- A feldolgozott adatokat átadja a grafikus komponensnek.

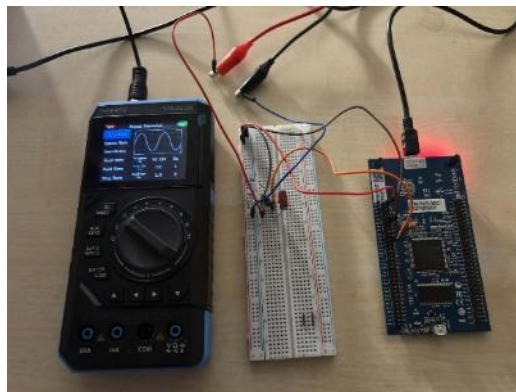
Eredmények

Az elkészült projekt vegyesen tartalmaz C és C++ forráskódot. A program RTOS módban fut, a GUI könyvtár implementációja miatt, de a perifériák interrupt-vezéreltek, és flageket állítanak. Az elkészült és flash-re írt program mérete ~900 kB.

Egy breadboard-on megépítettem a korábban mutatott egyszerű tranzisztoros áramkört (8. ábra), a tápellátást rákötöttem a *STM32F429-DISCI* kártya egyik „3V” jumper pinjére, a földelést az egyik „GND” pinre, illetve a 2N3904-es tranzisztor emitterét a „PA5”-ös pinre.

A jelgenerátor **pozitív** kivezetését rákötöttem a tranzisztor bázisára, a **negatív** kivezetést pedig a közös földelésre.

A teljesen összekapcsolt rendszer a jobb oldali ábrán látható. (9. ábra)



9. ábra - Az elkészült rendszer

Tesztelés különböző formájú jelekkel

Legelőször úgy néztem meg az *FFT*-t, hogy nem adtam be jelet. Az *FFT* vizualizációból látható, hogy az összerakott rendszer egy körülbelül -85 dB erősségű zajpadlóval rendelkezik. Egy kézi jelgenerátorral 1 V amplitúdójú tesztjeleket adtam be a rendszernek.

	Oscilloszkóp	FFT
Nincs bemenet		
Színusz (1 kHz)		
Négyszög-jel (1 kHz)		

Az FFT-kijelzőn a vártaknak megfelelő eredményeket láthatjuk.

- *Nincs bemenet*: Csak az áramkör megvalósítási mivoltjából származó alapzaj látszódik a képernyőn.
- *Színusz*: Az 1 kHz-es szinuszhullám az előző számításaink szerint a képernyő közepén jelenik meg egy darab magas csúcs formájában.
- *Négyszög-jel*: A csúcsok először az egy 1 kHz-es frekvenciánál jelennek meg, majd az 1 kHz páratlan számú felharmonikusainál.¹

Felhasznált szoftver és hardver

Szoftverek

- Fejlesztői környezet: *STM32CubeIDE 2.1.1*
- Hardver konfigurátor: *STM32CubeMX 6.17.0*
- FLASH programozó: *STM32CubeProgrammer 2.22.0*
- GUI keretrendszer: *TouchGFX 4.26.1*
- GUI szerkesztő: *TouchGFXDesigner 4.26.1*
- DSP segédkönyvtár: *X-CUBE-ALGOBUILD*

Hardverek

- *x86-architektúrájú laptop*
- *FNIRSI DST-210 multiméter/jelgenerátor*
- *STM32F429I-DISC1 fejlesztőkártya*

Elektronikai alkatrészek

- *Breadboard*
- *100nF fóliakondenzátor*
- *18k Ω 0.4W ellenállás*
- *51k Ω 0.4W ellenállás*
- *3k Ω 0.4W ellenállás*
- *2N3904 bipoláris NPN tranzisztor*
- *Jumper kábelek*

¹ A négyszög-jelek csak az alapharmonikus páratlan számú többszöröseiből tevődnek össze.